

Classifying Handwritten Digits with K-Means

1st Spencer Koh
Dept. of Elec. and Systems Eng.
Washington University in St. Louis
St Louis, USA
koh.s@wustl.edu

2nd Adan Mendoza
Dept. of Elec. and Systems Eng.
Washington University in St. Louis
St Louis, USA
adanm@wustl.edu

3rd Carlos Paiz
Dept. of Elec. and Systems Eng.
Washington University in St. Louis
St Louis, USA
paiz@wustl.edu

Abstract—This case study applies k-means clustering to sort handwritten digits. We tested k values from 20 to 100, where k is the number of clusters, and compared accuracy and runtime. To improve on random initialization, we designed our own method for selecting centroids that favor points farther away than those already chosen, reducing the chance of centroids starting too close together. In the end, we decided that the value of $k = 28$ gave the best results given its cost. At $k = 28$ our algorithm correctly classified around 73% of the images. Since our algorithm separated the digits in the testing set fairly well based only on the data from the training set, similar clustering methods could be useful in applications where there is a sufficient sample of data that can be used to classify new data, measuring the same things.

I. INTRODUCTION

This project has helped us enrich our knowledge of ESE 1050 topics by combining everything we have learned, such as vector operations, norms, and clustering, through a real world application. Our goal was to implement our own version of the k-means clustering algorithm to classify handwritten digits in an efficient and accurate manner. By creating our own clustering algorithm, we have gained a clearer understanding of how k-means clustering works and how it can be applied to classify new data based on past data. Additionally, we tested a large range of k values to identify the best trade-off between computational power and classification accuracy, and we implemented a way to identify outliers that did not fit well in their assigned cluster.

II. METHODS

A. Implementing K-Means

We changed the way our centroids were initialized to favor picking data points that were further away from the previously established centroids. This was to avoid initializing with centroids that start too close together and risk them ending up too similar. We accomplished this by weighting each vector based on its minimum distances to the other centroids so that they would have a higher chance of getting picked as the next centroid.

Then we assigned each image vector the centroid with the lowest Euclidean distance and repeated that for all of the image vectors. Then we updated the centroids to be the average of all of the vectors in their respective clusters. Lastly, we repeated those steps until updating the centroids wouldn't change them from where they currently are.

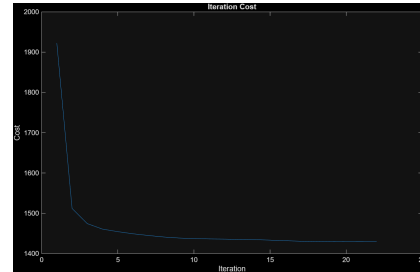


Fig. 1: Cost per iteration of k-means

B. Classifying Centroids

Since we know what number each image corresponds to, we took the most common number in each cluster and assigned that value to that cluster's centroid. We did this automatically in Matlab instead of by hand to eliminate having to guess when a centroid could be two different numbers. For example if the cluster contains enough threes, fives, and eights it could be hard to tell which one there are more of.

C. Choosing K Value and Max Iterations

To choose our k value we iterated through our k-means clustering algorithm with k values ranging from 20 to 100. In each iteration we recorded its accuracy and runtime. We decided that a k around 28 provided a good balance of accuracy while not taking too long to run. Since accuracy is increasing logarithmically and runtime is increasing linearly even though higher k values can provide higher accuracy we decided they were not worth it. We chose a max iterations of 100, though our k-means algorithm would stop early if updating the centroids would not change them, which was almost always the case.

III. RESULTS AND DISCUSSION

A. Results & Outliers

Our algorithm classified 73% of images in the test set correctly. We defined outliers as any vector that's distance to the centroid was $\geq 30\%$ more than the average. With that definition there were only 5 outliers in the testing data set.

B. Misclassifications

The most commonly misclassified digits were nine and four. Some of the misclassifications can be due to the images being

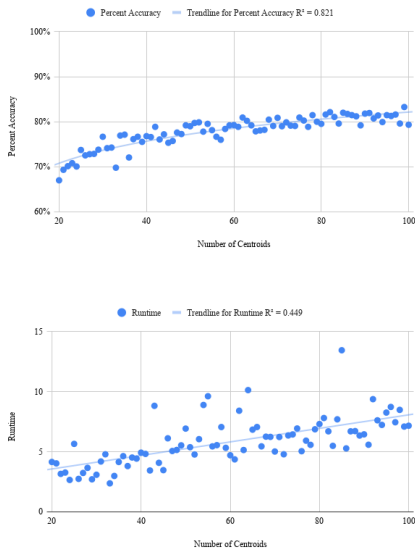


Fig. 2: Effect of increasing the number of centroids on classification accuracy and runtime.

bad such as one three and two nines. Some of the misclassified numbers are messy such as the three, three with a bubble at the bottom, the nine that doesn't really have a hole, the six where the hole is abnormally small, and the eight that has a bar like a five.

IV. CONCLUSION

We found that k-means clustering is a somewhat reliable algorithm for digit classification. With 28 clusters we were able to obtain around 73% accuracy and if we weren't also trying to limit runtime we could have raised the accuracy. One thing to note is that if you increase the number of centroids such that there are more centroids than vectors you are testing you are no longer really clustering the data. Something that could be tested to try to improve the algorithm would be to weight some entries in each vector that would typically

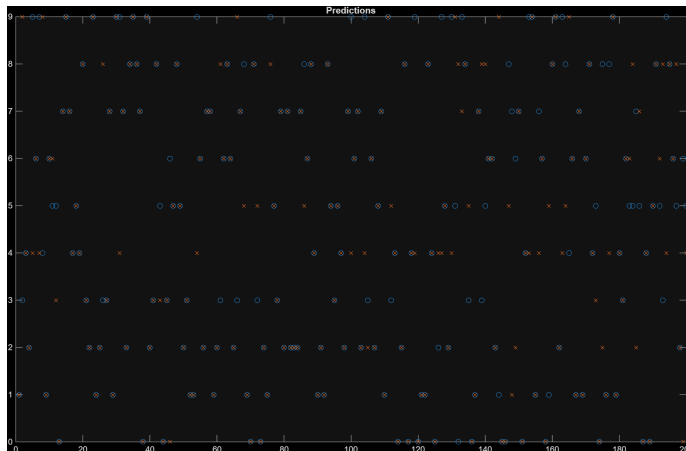


Fig. 3: Classification of digits

correspond to some unique features of numbers. Instead of using the weighted distances to initialize the centroids an algorithm that spreads them onto different numbers evenly or puts centroids on numbers that look more distinct could yield higher accuracy.

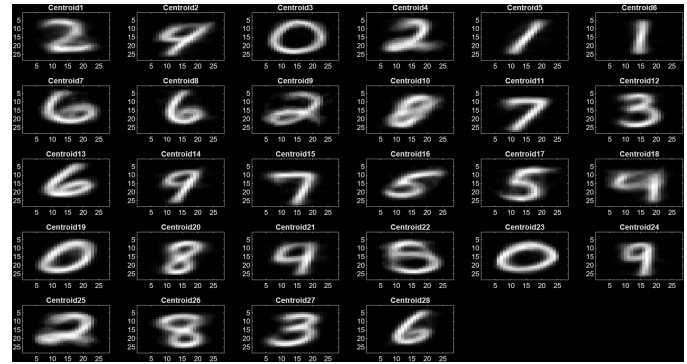


Fig. 4: Centroids

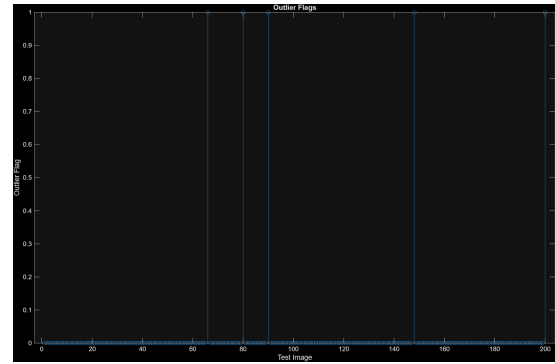


Fig. 5: Outliers



Fig. 6: Misclassified digits